

README

Part 12: 微调实战 — LLaMA-Factory (LoRA / QLoRA / DPO 全流程)

- > Part 8 用几十行手写了 LoRA 的原理；本部分把同样的技能放大到工业工具：
- > 用 LLaMA-Factory 在真实 7B 模型上走完 LoRA SFT → QLoRA → 合并导出 → DPO 的完整工作流。
- > 学完你能独立承担"把一个开源基座调到业务任务上"的工程任务。
- > 主源：[hiyouga/LlamaFactory](https://github.com/hiyouga/LlamaFactory) (74.4k, Apache-2.0)

学习目标

完成本部分后，你将能够：

- 理解 微调在 LLM 链路中的位置和价值
- 手写 LoRA SFT 的完整流水线（chat template + masking + LoRA 注入 + 训练）
- 解释 LoRA/QLoRA/DPO 的数学原理和工程权衡
- 配置 LLaMA-Factory 的 yaml 并完成 LoRA SFT → QLoRA 7B → export → DPO 的生产链路
- 识别 微调中的常见陷阱并设计防范策略

章节导航

	内容	对应
手写 LoRA SFT：工具自动化的到底是什么](01_handwritten_sft_lora.md)	chat template+masking+LoRA 注入+训练循环 手写一遍（工具的"内部透视"）	'01'
LLaMA-Factory 工作流](02_llamafactory_workflow.md)	identity LoRA SFT → WebUI → QLoRA 7B → export 合并 → DPO-LoRA	—

前置知识

- 必须掌握：
- [Part 8 08 章](../Part8_post_training/tutorial/08_lora_and_classification.md)：从零 LoRA（A/B 初始化、 α/r 、注入位置）——本章工具的每个 yaml 字段都对应它
 - [Part 8 02 章](../Part8_post_training/tutorial/02_sft_and_chat.md)：SFT 与 prompt masking
 - [Part 8 03 章](../Part8_post_training/tutorial/03_reward_and_dpo.md)：DPO
- 建议掌握：
- [Part 10](../Part10_distributed/tutorial/README.md)：FSDP 与多卡基础（多卡微调用到）
- 可选：
- [Part 11](../Part11_alignment_verl/tutorial/README.md)：verl 对齐实战（DPO 与 GRPO 的对比）

在 LLM 链路中的位置

预训练(Part 7/13) → [本部分: 微调 SFT/LoRA/QLoRA/DPO] → 对齐 RL(Part 11) → 部署(Part 14)
↑
你在这里

为什么微调是"业务可用"的第一手段：

证据	说明
成本	全参微调 7B 需要 ~120GB 显存，QLoRA 只需 ~6GB
效果	LoRA 在大部分任务上能达到全参微调 90%+ 的效果
速度	QLoRA 7B 在 4090 上 1-2 小时可完成
生态	LLaMA-Factory 支持 100+ 模型、多种微调方法

微调是把基座变成"业务可用"的第一手段；LLaMA-Factory 是这条路上最流行的统一工具
(一个 yaml 覆盖 100+ 模型 / 全参+LoRA+QLoRA+DoRA+GaLore / SFT+RM+DPO+KTO+ORPO)。

理论背景

问题引入：为什么需要微调？

预训练模型虽然强大，但有两个根本限制：

- 1. 知识截止：预训练数据有截止日期，不知道最新信息
- 2. 任务适配：预训练目标是"预测下一个 token"，不是"回答问题"或"遵循指令"

微调（Fine-Tuning）通过在特定任务数据上继续训练来弥补：

预训练: "学习语言和知识" → 通用能力
微调: "学习特定任务和格式" → 任务适配
对齐: "学习人类偏好" → 安全和有用

> 类比：预训练像是读完大学，微调像是入职培训。大学教你通用知识，
> 但入职培训教你如何在这家公司工作。

数学推导：LoRA 的低秩分解

LoRA（Low-Rank Adaptation）的核心思想是：用低秩矩阵近似权重更新。

问题设定：

- 预训练权重： $W \in \mathbb{R}^{d \times k}$
- 全参微调更新： $\Delta W \in \mathbb{R}^{d \times k}$
- LoRA 更新： $\Delta W = B \times A$ ，其中 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$

推导过程：

Step 1: 全参微调的权重更新

$$W' = W + \Delta W$$

$$\Delta W \text{ 的参数量} = d \times k$$

Step 2: LoRA 的低秩分解

$$\Delta W = B \times A$$

B 的参数数量 = $d \times r$

A 的参数数量 = $r \times k$

总参数量 = $r \times (d + k)$

Step 3: 参数量对比

全参: $d \times k$

LoRA: $r \times (d + k)$

压缩比 = $(d \times k) / (r \times (d + k)) = d \times k / (r \times (d + k))$

示例：d=4096, k=4096, r=8

全参: $4096 \times 4096 = 16,777,216$

LoRA: $8 \times (4096 + 4096) = 65,536$

压缩比: 256 倍

性质：

- LoRA 不增加推理延迟（合并后 $W' = W + (\alpha/r) \cdot BA$ ）
- LoRA 的 A 用高斯初始化，B 用零初始化（训练开始时 $\Delta W = 0$ ）
- α/r 是缩放因子，控制 LoRA 的"学习强度"

历史脉络：微调方法演进

2018: 全参微调（Full Fine-Tuning）

↓ 显存开销大

2019: Adapter（Houlsby et al.）

↓ 增加推理延迟

2021: LoRA（Microsoft）

↓ 不增加推理延迟

2023: QLoRA（Dettmers et al.）

↓ 4bit 量化 + LoRA

2024: DoRA/GaLore/...

↓ 更多优化方法

关键论文：

- LoRA: [LoRA: Low-Rank Adaptation of Large Language Models](<https://arxiv.org/abs/2106.09685>)
- QLoRA: [QLoRA: Efficient Finetuning of Quantized LLMs](<https://arxiv.org/abs/2305.14314>)
- DPO: [Direct Preference Optimization](<https://arxiv.org/abs/2305.18290>)

环境与版本策略

`bash`

独立 venv（不污染课程主环境）；策略：跟随 latest（耦合轻）

`uv venv .venv-lf && source .venv-lf/bin/activate`

`git clone https://github.com/hiyouga/LlamaFactory && cd LlamaFactory`

```
pip install -e ".[torch,metrics]" # python ≥3.11 ; flash-attn 可选（装不上可跳过）
llamafactory-cli version # 验证
```

硬件	可做什么（官方文档数字）
CPU	identity 小模型 LoRA 演示（脚本 01 的手写版无需任何安装）
1×24GB（4090）	**QLoRA 7B（官方 4bit=6GB）**、LoRA bf16 7B（16GB）、DPO-LoRA
多卡	`FORCE_TORCHRUN=1` 起 DDP / DeepSpeed ZeRO-3（见 Part 10 知识）

学习地图（由点到面）

手写 LoRA SFT（脚本01：看得见每一行） ← 点
↓ "这几百行被 yaml 的哪几个字段自动化了？"
LLaMA-Factory 最小闭环（identity 0.5B） ← 线
↓ 真实模型 + 真实数据
QLoRA 7B → export 合并 → chat/api ← 面
↓ 偏好对齐
DPO-LoRA（呼应 Part 8 03 章） → 面试/工作就绪

课后作业

每章末尾有思考题（<details> 折叠答案）。全部学完后：
[Assignment 12](../../assignments/assignment_12/)

相关资源

- [LLaMA-Factory](https://github.com/hiyouga/LlamaFactory)（官方 docs 与 examples/yaml 是最好的教程）
- [unsloth](https://github.com/unslothai/unsloth)（75.2k，单卡加速微调，免费 Colab notebook 丰富——作业对照用）
- [huggingface/peft](https://github.com/huggingface/peft)（LoRA 底层库）
- [LoRA 论文](https://arxiv.org/abs/2106.09685) · [QLoRA 论文](https://arxiv.org/abs/2305.14314)

[← 上一章：Part 11 verl 对齐实战](../../Part11_alignment_verl/tutorial/README.md) | [下一章：Part 13 数据工程 →](../../Part13_data_engineering/tutorial/README.md)

01_handwritten_sft_lora

01 — 手写 LoRA SFT：LLaMA-Factory 自动化的到底是什么

> 工具的价值只有在你知道它替你做了什么时才能兑现。本章用 ~250 行把

> LLaMA-Factory 一个 yaml 背后的完整流水线手写一遍（跑 [scripts/01_handwritten_sft_lora.py](../scripts/01_handwritten_sft_lora.py)），
> 然后给出逐字段对照表——之后看任何微调 yaml，你都能指出"每个字段对应哪几行代码"。

学习目标

完成本章后，你将能够：

- 手写 LoRA SFT 的完整流水线（5 个步骤）
- 解释 每个步骤对应的 LLaMA-Factory yaml 字段
- 识别 "注入后忘搬 device" 等常见陷阱
- 说出 手写版与工具版的真实差距

前置知识

必须掌握：

- Part 8 08 章：LoRALinear 的 A/B 初始化与 α/r （本章直接复用）
- Part 8 02 章：prompt masking (labels=-100)

理论背景

问题引入：为什么需要 LoRA 而不是全参微调？

全参微调（Full Fine-Tuning）虽然效果好，但有两个根本限制：

1. 显存开销大：7B 模型全参微调需要 ~120GB 显存（参数 + 梯度 + 优化器）
2. 容易过拟合：小数据集上全参微调容易"忘记"预训练知识

LoRA（Low-Rank Adaptation）通过只训练低秩矩阵来弥补：

全参微调: $W' = W + \Delta W$ # $\Delta W \in \mathbb{R}^{d \times k}$, 参数量 = $d \times k$

LoRA: $W' = W + (\alpha/r) \cdot BA$ # $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, 参数量 = $r \times (d+k)$

示例：d=4096, k=4096, r=8

全参: 16,777,216 参数

LoRA: 65,536 参数（压缩 256 倍）

- > 类比：全参微调像是重新装修整栋房子，LoRA 像是只换几件家具。
- > 效果差不多，但成本低很多。

数学推导：LoRA 的初始化和缩放

问题设定：

- 预训练权重： $W \in \mathbb{R}^{d \times k}$
- LoRA 矩阵： $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$
- 缩放因子： α （学习强度）、 r （秩）

推导过程：

Step 1: 初始化

$A \sim N(0, \sigma^2)$ # 高斯初始化

$B = 0$ # 零初始化

性质：训练开始时 $\Delta W = BA = 0$ ，不改变预训练权重

Step 2: 前向传播

$h = Wx + (\alpha/r) \cdot BAx$

其中：

- Wx 是预训练的输出
- $(\alpha/r) \cdot BAx$ 是 LoRA 的增量

Step 3: 合并（推理时）

$W' = W + (\alpha/r) \cdot BA$

性质：合并后推理零额外开销

关键洞察：

- α/r 控制 LoRA 的"学习强度"： α 越大，LoRA 影响越大
- r 越大，LoRA 的表达能力越强，但参数也越多
- 实践中 $r=8-64$ ， $\alpha=2r$ 是常见配置

代码实现

微型管线五步（玩具任务："回声指令"——回应复述指令的第一个词）

运行 `[scripts/01_handwritten_sft_lora.py](../scripts/01_handwritten_sft_lora.py)` 验证以下步骤。

[0] 基座预热 —— "预训练过的"玩具基座（结构就绪即可）

[1] LoRA 注入 —— 4 层 MLP Linear，可训练 6,144/200,664（3.1%）

[2] SFT 训练 —— loss 3.572 $\rightarrow$ 0.076（chat 格式 + 任务映射都学会了）

[3] 推理验证 —— chat 格式 3/3 正确；回声任务 2/3（400 步玩具训练的

正常欠拟合——真任务上这个位置由更多数据/步数兜底）

[4] 合并（merge） —— BA 并回 W（精确加法），同一批 prompt 前后行为一致，零额外开销

> 以上为脚本真实输出（RTX 4090 / CPU 均可复现，~3 秒）。

形状追踪：LoRA 注入过程

口径说明：下图与脚本同口径（`apply_lora(model, r=4, alpha=8.0)`），数字可直接对上脚本 [1] 的输出——每层 $1,536 \times$ 注入 4 层（2 个 Block $\times$ MLP 两个 Linear）= 6,144。

LoRA 注入过程（以 MLP 的第一个 Linear 为例， $r=4$, $\alpha=8$ ）
原始层: <code>Linear(in_features=96, out_features=288)</code>

权重 W: (288, 96)
注入后:
LoRALinear(
base_layer: Linear(96, 288) # 冻结
lora_A: (4, 96) # 可训练, $A \sim N(0,1)/\sqrt{r}$
lora_B: (288, 4) # 可训练, B 零初始化
scaling: $\alpha/r = 8/4 = 2.0$ # 缩放因子
)
前向传播:
x: (batch, seq_len, 96)
↓ base_layer
base_out: (batch, seq_len, 288)
↓ lora_A (x @ A.T: ($\cdots$, 96) @ (96, 4))
lora_out: (batch, seq_len, 4)
↓ lora_B ($\cdots$ @ B.T: ($\cdots$, 4) @ (4, 288))
lora_out: (batch, seq_len, 288)
↓ scaling (α/r)
lora_out: (batch, seq_len, 288) * 2.0
↓ addition
output: base_out + lora_out
可训练参数: $4 \times 96 + 288 \times 4 = 384 + 1152 = 1,536$ (本层)
原始参数: $288 \times 96 = 27,648$
压缩比: $27,648 / 1,536 = 18$ 倍
全模型: $1,536 + (\text{MLP 第二个 Linear 同为 } 1,536) = 3,072/\text{Block}$
$\times 2$ 个 Block = 6,144 —— 正是脚本 [1] 打印的可训练参数

五步与 yaml 字段的对照（本章核心产出）

手写函数	LLaMA-Factory yaml / CLI 字段
`build_sample()` 的 prompt 拼接 + `labels[:n_prompt]=-100`	`template:` + `train_on_prompt: false`
`apply_lora(r, alpha)` 注入 MLP Linear	`lora_target:` / `lora_rank:` / `lora_alpha:`
`make_sft_data()` 的 (instruction, response)	`dataset:` + `dataset_info.json` 的列映射
`sft_train()` 的 AdamW/lr/步数	`learning_rate:` / `num_train_epochs:` / `per_device_train_batch_size:`
`pad_batch()` 的右侧 padding + -100 填充	`cutoff_len:`（packing=多条样本拼进定长序列免 padding 浪费，与 padding 二选一）
`merge_lora()` 的 $W += (\alpha/r) \cdot BA$	`llamafactory-cli export`

- 读 yaml 的新能力：**lora_target: all** = "所有 Linear 都注入"；**lora_dropout** 是 BA 旁路上的 dropout；**train_on_prompt: true** = 把 masking 撤掉（Part 8 02 章

讲过为什么不这么做）。

手写版 vs 工具版的真实差距

维度	手写（本脚本）	LLaMA-Factory
数据	20 条玩具指令，内存 list	100+ 数据集目录、json/jsonl/sharegpt 格式自动识别
模型	200K 玩具 GPT	100+ HF 模型开箱即用（Qwen/Llama/GLM/DeepSeek…）
训练技巧	固定 lr、无 warmup/梯度累积/断点续训	全部内置（对应附录 D 的"bells & whistles"）
量化	无	QLoRA 4bit（NF4）一行开关
多卡	无	DDP/ZeRO-3/FSDP 一键（FORCE_TORCHRUN=1）
产物	内存里的权重	checkpoint + `export` 合并 + `chat`/`api` 部署

• 结论：手写教会你"字段<->代码"的映射，工具给你工程完备性。两者都过一遍，你就同时具备"改得动工具"（debug、自定义）和"用得对工具"（选字段、估显存）的能力。

工程实践

调试展示：常见错误与修复

错误 1：注入后忘搬 device

症状：

```
RuntimeError: Expected all tensors to be on the same device
```

原因：LoRA 新建的 A/B 参数默认在 CPU，注入之后必须再 `.to(device)`

解法：

```
python
```

注入 LoRA

```
apply_lora(model, r=8, alpha=16)
```

必须搬回 GPU ！

```
model = model.to(device)
```

错误 2：dtype 不一致

症状：

```
RuntimeError: expected scalar type Half but found Float
```

原因：模型是 fp16，但 LoRA 参数是 fp32

解法：

```
`python
```

注入 LoRA 后统一 dtype

```
model = model.to(device, dtype=torch.float16)
```

错误 3：梯度未正确 mask

症状：loss 不下降，或 loss 为 0

原因：labels 没有正确设置 -100（prompt 部分不应该计算 loss）

解法：

```
`python
```

确保 prompt 部分的 labels 是 -100

```
labels[:n_prompt] = -100
```

性能数据（实测参考）

模型	方法	可训练参数	显存占用	训练时间	效果
200K 玩具	LoRA r=4	6,144 (3.1%)	~1GB	<1min	2/3 正确
7B	全参微调	7B (100%)	~120GB	~10h	基准
7B	LoRA r=8	~20M (0.3%)	~16GB	~2h	~95% 全参效果
7B	QLoRA r=8	~20M (0.3%)	~6GB	~2h	~93% 全参效果

- > 数据来源：LLaMA-Factory 官方 benchmark + 本课开发机实测
- > （RTX 4090, torch 2.6.0+cu124；玩具行 = 脚本 01 复跑，7B 行 = 官方量级参考）

常见陷阱

陷阱 1：数据格式不匹配

症状：训练时 loss 不下降，或输出格式混乱

原因：数据格式与模型的 chat template 不匹配

解法：检查数据格式是否符合模型的 chat template（如 Qwen 用 `<|im_start|>`）

陷阱 2：显存不足 (OOM)

症状：CUDA out of memory

原因：cutoff_len 太大，或 batch_size 太大

解法：减小 cutoff_len（如 512）或 batch_size（如 1）

陷阱 3：LoRA rank 选择不当

症状：效果不好，或显存不足

原因：rank 太小（表达能力不足）或太大（显存开销大）

解法：从 r=8 开始，根据效果和显存调整

最佳实践

LoRA 配置推荐

参数	推荐值	说明
`lora_rank`	8-64	从 8 开始，根据效果调整
`lora_alpha`	$2 \times \text{lora_rank}$	常见配置
`lora_target`	all	注入所有 Linear
`lora_dropout`	0.05-0.1	防止过拟合
`learning_rate`	$1\text{e-}4 \sim 5\text{e-}5$	LoRA 学习率通常比全参大

QLoRA 配置推荐

参数	推荐值	说明
`quantization_bit`	4	NF4 量化
`double_quantization`	true	双重量化，省常数开销
`quantization_type`	nf4	NormalFloat4 格式

学完本章你能...

- 说出 LoRA SFT 流水线的五个环节及各自对应的 yaml 字段
- 解释 merge 之后为什么推理零开销
- 警觉"注入后忘搬 device"这类扩展模块的经典 bug
- 画出 LoRA 注入的形状追踪图
- 识别数据格式不匹配、显存不足等常见陷阱

概念检验

<details>
<summary>Q1: 为什么本脚本严格冻结 lm_head？放开它（全参 + LoRA 混合）会有什么变化？</summary>
A: 放开 lm_head 是常见的"LoRA + 部分全参"混合档位（embed/lm_head 可训练），任务适配通常更快，但可训练参数与优化器显存随之上升。LlamaFactory 有 **additional_target: trainables** 字段控制这一档。玩具实验：放开 lm_head 后 400 步内 acc 应更稳。
</details>
<details>
<summary>Q2: 如果把 lora_rank 从 4 提到 64，可训练参数变成多少？什么时候值得？</summary>
A: 每层从 $r \cdot (d+k)=4 \cdot (96+288) \times 2$ 变为 $64 \cdot 384 \times 2$ 的量级——参数比例从 3.1% 升到两位数。

值得的场景：任务与预训练分布差距大、或数据量大（几十万条以上）；小任务 $r=8/16$ 通常够。

</details>

<details>

<summary>Q3: LoRA 的 A 和 B 分别用什么初始化？为什么这样设计？</summary>

A: A 用高斯初始化，B 用零初始化。这样训练开始时 $\Delta W = BA = 0$ ，不改变预训练权重。随着训练进行，B 逐渐非零，LoRA 开始"学习"任务特定的权重更新。

</details>

动手实践

<details>

<summary>练习 1: 实现 LoRA 注入函数</summary>

任务：实现一个函数，给模型的 Linear 层注入 LoRA。

验收标准：

- ☐ 正确注入 lora_A 和 lora_B
- ☐ 正确设置 $\text{scaling} = \alpha / r$
- ☐ 冻结原始权重
- ☐ 返回可训练参数数量

步骤提示：

```
`python
def apply_lora(model, r=8, alpha=16):
    """
```

Steps:

1. 遍历模型的所有模块
2. 找到 Linear 层
3. 替换为 LoRALinear
4. 冻结原始权重
5. 返回可训练参数数量

```
"""
```

TODO: Implement

```
pass
`
```

</details>

<details>

<summary>练习 2: 实现 LoRA 合并函数</summary>

任务：实现一个函数，将 LoRA 权重合并回原始权重。

验收标准：

- ☐ 正确计算 $W' = W + (\alpha / r) \cdot BA$
- ☐ 合并后删除 LoRA 参数
- ☐ 合并后推理结果不变

步骤提示：

```
`python
def merge_lora(model):
    """
```

Steps:

1. 遍历模型的所有模块
2. 找到 LoRALinear 层
3. 计算 $W' = W + (\alpha / r) \cdot BA$
4. 替换原始权重
5. 删除 LoRA 参数

"""

TODO: Implement

pass

`

</details>

<details>

<summary>练习 3: 实现显存估算函数</summary>

任务：实现一个函数，估算 LoRA 微调的显存占用。

验收标准：

- [] 输入：模型参数量、LoRA rank、batch_size、seq_len
- [] 输出：预估显存占用（GB）
- [] 考虑参数、梯度、优化器状态

步骤提示：

`python

def estimate_lora_memory(

model_params_B: float, # 模型参数量（单位：B）

lora_rank: int,

batch_size: int,

seq_len: int,

) -> float:

"""

估算 LoRA 微调的显存占用

经验公式：

- 参数: $\text{model_params_B} * 2 \text{ bytes (fp16)}$
- 梯度: $\text{model_params_B} * 2 \text{ bytes (fp16)}$
- 优化器: $\text{model_params_B} * 8 \text{ bytes (Adam)}$
- 激活: $\text{batch_size seq_len d_model} * 4 \text{ bytes}$

Steps:

1. 计算参数显存
2. 计算梯度显存
3. 计算优化器显存
4. 计算激活显存
5. 汇总

"""

TODO: Implement

pass

`

</details>

课后作业

完成本章后，去 Assignment 12 完成练习：

[Assignment 12](../../assignments/assignment_12/)

下一步

同样的流程交给工具：identity 数据集 → WebUI → QLoRA 7B → export → DPO-LoRA。

[02 — LLaMA-Factory 工作流](02_llamafactory_workflow.md)

02_llamafactory_workflow

02 — LLaMA-Factory 工作流：identity → QLoRA 7B → export → DPO-LoRA

- > 手写完成（01 章），现在把同一流程交给工具，规模放大到 7B 真实模型。
- > 本章是一条可直接照抄的命令流水线（每步标注预期产物与耗时，4090 实测量级），
- > 环境按 README 的版本策略：独立 venv 跟随 LLaMA-Factory latest。

学习目标

完成本章后，你将能够：

- 配置 LLaMA-Factory 的 yaml 文件并理解每个字段的含义
- 完成 LoRA SFT → QLoRA 7B → export → chat/api 的生产链路
- 用 "官方显存数字 + Part 10 账本" 预估自己的训练能不能跑
- 用工具跑 DPO-LoRA 并读懂 rewards/margins 曲线
- 把任何微调 yaml 翻译成 "五步管线" 来 debug

前置知识

必须掌握：

- 01 章：五步管线与 yaml 字段映射（本章每个 yaml 字段都引用它）
- Part 8 03 章：DPO（本章用工具跑一遍）

理论背景

问题引入：为什么需要 QLoRA？

LoRA 虽然已经很省显存，但 7B 模型的底座权重仍然需要 ~14GB（fp16）。
QLoRA 通过量化底座权重来进一步节省显存：

```
`  
LoRA: 底座 fp16 (14GB) + LoRA bf16 (~20MB) = ~14GB  
QLoRA: 底座 4bit (3.5GB) + LoRA bf16 (~20MB) = ~3.5GB  
`
```

- > 类比：LoRA 像是只换几件家具，QLoRA 像是把家具换成折叠的。
- > 平时折叠起来省空间，用的时候展开。

数学推导：QLoRA 的量化过程

问题设定：

- 底座权重： $W \in \mathbb{R}^{d \times k}$ (fp16, 每个参数 2 字节)
- 量化后： $W_q \in \mathbb{R}^{d \times k}$ (4bit, 每个参数 0.5 字节)

推导过程：

```
`  
Step 1: fp16 存储  
每个参数 2 字节  
7B 模型 =  $7 \times 10^9 \times 2 = 14\text{GB}$   
`
```

```
Step 2: 4bit 量化  
每个参数 0.5 字节  
7B 模型 =  $7 \times 10^9 \times 0.5 = 3.5\text{GB}$   
`
```

```
Step 3: 双重量化  
量化常数也量化（每 64 个参数共享一个量化常数）  
额外节省 ~0.37GB  
总计:  $\sim 3.5\text{GB} + 0.37\text{GB} \approx 3.87\text{GB}$   
`
```

性质：

- NF4 (NormalFloat4) 是专门为正态分布设计的 4bit 格式
- 双重量化 (Double Quantization) 把量化常数也量化，进一步节省显存
- LoRA 的 A/B 保持 bf16 训练，不被量化

代码实现

0. 环境（一次性）

```
`bash  
uv venv .venv-lf && source .venv-lf/bin/activate  
git clone https://github.com/hiyouga/LlamaFactory && cd LlamaFactory  
pip install -e ".[torch,metrics]"  
llamafactory-cli version # 能打印版本即 OK  
`
```

1. 最小闭环：identity LoRA SFT（小模型，小时级内出结果）

LLaMA-Factory 自带 **identity** 数据集（教模型“我是谁”），最适合第一次跑通：

```
`bash
```

官方示例 yaml：examples/train_lora/qwen_lora_sft.yaml
改两行即可

```
llamafactory-cli train \  
--model_name_or_path Qwen/Qwen2.5-0.5B-Instruct \  
--dataset identity,alpaca_gpt4_zh \  
--template qwen --finetuning_type lora \  
--lora_target all --lora_rank 8 --lora_alpha 16 \  
--output_dir saves/qwen05-identity --per_device_train_batch_size 4 \  
--learning_rate 5e-5 --num_train_epochs 3.0 --plot_loss true
```

⚠️ yaml 字段以你安装版本的 examples/
实际文件名为准（仓库迭代快，

例如 qwen_lora_sft.yaml 在新版已更名 qwen3_lora_sft.yaml)

对照 01 章五步：--template = build_sample；--lora_target all = 注入所有 Linear；
--train_on_prompt 默认 false = prompt masking。产物：saves/qwen05-identity/（adapter
权重 + loss 图）。

2. WebUI：LLaMA Board（建立配置直觉）

```
`bash  
llamafactory-cli webui # 浏览器打开，零代码配置并启动训练
```

用途不是生产训练，而是把字段玩一遍：改 lora_rank/cutoff_len/learning_rate 时
页面会实时估算显存——把 01 章的手写账本和 GUI 的估算互相印证。

3. QLoRA 7B（4090 主菜，官方数字：4bit 7B $\approx$ 6GB）

```
`bash  
llamafactory-cli train examples/train_qlora/qwen3_lora_sft_otfq.yaml
```

（文件名以安装版本 examples/ 为准；关键这 4
个字段——对照手写版"缺的量化"）：

quantization_bit: 4 $\leftarrow$ NF4 底座（QLoRA 的 Q；NF4=4-bit
NormalFloat 网格量化格式）

finetuning_type: lora $\leftarrow$ 只训 BA

double_quantization: true $\leftarrow$ 双重量化：把每组的量化常数
scale 再量化一遍，省常数开销

⚠ 记得加 --output_dir saves/qwen7b-qlora (§ 4 export 要用这个路径)

预期：7B 模型 + batch 1-2，显存 6-10GB (4090 余量充足)，10K 条数据 1-2 小时量级。
观察点：nvidia-smi 里权重本体常驻 ~4GB (4bit)，训练波动部分来自梯度/优化器——只有 BA 有梯度，这正是 Part 8 08 章"LoRA 省的是优化器+梯度"的实证。

4. 合并与部署

```
`bash
llamafactory-cli export --model_name_or_path Qwen/Qwen2.5-7B-Instruct \
--adapter_name_or_path saves/qwen7b-qlora --export_dir models/qwen7b-merged \
--export_size 4 --export_legacy_format false
```

合并 = 手写版的 $W += (\alpha / r) \cdot BA$ (01 章 merge_lora) ；之后是普通模型：

```
llamafactory-cli chat --model_name_or_path models/qwen7b-merged
llamafactory-cli api --model_name_or_path models/qwen7b-merged # OpenAI 兼容服务
```

5. DPO-LoRA (偏好对齐, 呼应 Part 8 03 章)

```
`bash
llamafactory-cli train examples/train_lora/qwen3_lora_dpo.yaml # 文件名以版本为准
```

数据: UltraFeedback 的 (prompt, chosen, rejected) 三元组 (Part 8 03 章同款语义)

关键字段: pref_beta: 0.1 (= DPO 的 β)、pref_loss: sigmoid (标准 DPO)

预期现象 (记录进面经)：DPO 后 rewards/chosen 上升、rewards/margins 变正且扩大；lr 用 5e-6 量级 (比 SFT 更小——Part 7 05 章"越靠后 lr 越小"规律的又一实证)。

6. 手写 vs 工具：一张总账

能力	01 章手写	LLaMA-Factory
模型规模	200K 玩具	7B (QLoRA 6GB) / 100+ 模型
数据	20 条内存 list	100+ 数据集 + 自定义 json/sharegpt
量化	无	4bit NF4 (QLoRA) 一行
对齐	—	DPO/KTO/ORPO/RM 全家桶

能力	01 章手写	LLaMA-Factory
多卡	无	DDP/ZeRO-3 (FORCE_TORCHRUN)
部署	内存权重	export 合并 + chat/api

> ⚠ 工具不是魔法：跑挂时 90% 的问题在数据格式（template 不匹配、字段名不对）与
> 显存估算（cutoff_len × batch）。这两个 debug 能力恰恰来自 01 章的手写对照。

工程实践

调试展示：常见错误与修复

错误 1：数据格式不匹配

症状：

```
ValueError: Template qwen does not exist
```

原因：template 名称不对，或模型不支持该 template

解法：

```
`bash
```

查看支持的 template：WebUI（llamafactory-cli webui）的
template 下拉列表，

或官方 README 指定的
src/llamafactory/extras/constants.py（完整清单）

使用正确的 template

```
--template default # 或 auto
```

错误 2：显存不足 (OOM)

症状：

```
CUDA out of memory. Tried to allocate 2.00 MiB
```

原因：cutoff_len 太大，或 batch_size 太大

解法：

```
`bash
```

减小 cutoff_len

```
--cutoff_len 512
```

减小 batch_size

```
--per_device_train_batch_size 1
```

启用 gradient checkpointing

```
--gradient_checkpointing true
```

错误 3：数据集不存在

症状：

```
ValueError: Dataset xxx does not exist
```

原因：数据集名称不对，或未注册

解法：

```
`bash
```

查看支持的数据集：WebUI 的 dataset 下拉列表，

或仓库

data/dataset_info.json（内置数据集清单都登记在这里）

注册自定义数据集

在 dataset_info.json 中添加数据集定义

错误 4：export 失败

症状：

```
Error: adapter_name_or_path does not exist
```

原因：adapter 路径不对，或训练未完成

解法：

```
`bash
```

检查 adapter 是否存在

```
ls saves/qwen7b-qlora/
```

确保训练完成后再 export

性能数据（实测参考）

模型	方法	显存占用	训练时间	效果	来源
0.5B	LoRA r=8	~4GB	~10min	identity 学会	本机实测
7B	LoRA bf16 r=8	~16GB	~2h	~95% 全参效果	官方量级
7B	QLoRA 4bit r=8	~6GB	~2h	~93% 全参效果	官方量级
7B	DPO-LoRA	~8GB	~3h	rewards/margins 改善	官方量级

- > 数据来源：LLaMA-Factory 官方 benchmark + 本课开发机实测
- > （RTX 4090, torch 2.6.0+cu124；0.5B 行 = 本机实测，7B 行 = 官方量级参考，未逐行本机复现）

常见陷阱

陷阱 1：数据格式不匹配

- 症状：训练时 loss 不下降，或输出格式混乱
- 原因：数据格式与模型的 chat template 不匹配
- 解法：检查数据格式是否符合模型的 chat template（如 Qwen 用 `<|im_start|>`）

陷阱 2：显存估算不准

- 症状：训练时 OOM，但估算应该够
- 原因：没有考虑激活值的显存开销
- 解法：使用 WebUI 的显存估算功能，或手动计算

陷阱 3：LoRA rank 选择不当

- 症状：效果不好，或显存不足
- 原因：rank 太小（表达能力不足）或太大（显存开销大）
- 解法：从 r=8 开始，根据效果和显存调整

最佳实践

配置推荐

参数	推荐值	说明
`lora_rank`	8-64	从 8 开始，根据效果调整
`lora_alpha`	$2 \times \text{lora_rank}$	常见配置
`learning_rate`	$1\text{e-}4 \sim 5\text{e-}5$	LoRA 学习率通常比全参大
`cutoff_len`	512-2048	根据任务调整

参数	推荐值	说明
`batch_size`	1-4	根据显存调整
`num_train_epochs`	3-5	根据数据量调整

调试流程

- 1. 先用小模型（0.5B）跑通流程
- 2. 检查数据格式是否正确
- 3. 检查显存是否足够
- 4. 逐步增大模型和数据量

学完本部分你能...

- 独立完成 LoRA SFT → QLoRA 7B → export → chat/api 的生产链路
- 用"官方显存数字 + Part 10 账本"预估自己的训练能不能跑
- 用工具跑 DPO-LoRA 并读懂 rewards/margins 曲线
- 把任何微调 yaml 翻译成"五步管线"来 debug
- 识别数据格式不匹配、显存不足等常见陷阱

概念检验

<details>
<summary>Q1: QLoRA 里"4bit"量化的到底是什么？LoRA 的 A/B 也被量化了吗？</summary>

A: 只量化冻结的底座权重（NF4 存储）；LoRA 的 A/B 保持 bf16/fp16 训练——
"4bit 底座 + 高精度小适配器"正是 QLoRA 的名字含义。这也是它省显存的来源：
 $7B \times 0.5B \approx 3.5GB$ 的底座 + MB 级的可训练部分。

</details>
<details>
<summary>Q2: export 合并时如果忘了先 CPU 化或 dtype 不一致会怎样？生产上为什么不合并的场景也存在？</summary>
A: dtype 不一致会静默精度损失或报错（fp16 底座 + bf16 BA 要先统一）。不合并的场景：
多租户动态切换适配器（vLLM multi-LoRA）——保留 adapter、按请求挂载更省显存。

</details>
<details>
<summary>Q3: DPO 的 rewards/margins 曲线怎么读？什么时候算"收敛"？</summary>

A: rewards/chosen 应上升，rewards/rejected 应下降，margins 应变正且扩大。
收敛标志：margins 稳定在 0.5-2.0 之间，不再明显波动。

</details>

动手实践

<details>
<summary>练习 1: 估算显存占用</summary>

任务：估算 7B 模型 QLoRA 微调的显存占用。

- 验收标准：
- [] 考虑参数、梯度、优化器状态

- [] 考虑 4bit 量化
- [] 结果与官方数字 (~6GB) 接近

步骤提示：

```
`python
def estimate_qlora_memory(model_params_B=7, lora_rank=8, batch_size=1, seq_len=512):
    """
```

Steps:

1. 计算底座权重显存 (4bit)
2. 计算 LoRA 参数显存 (bf16)
3. 计算梯度显存
4. 计算优化器显存
5. 计算激活显存
6. 汇总

```
"""
```

TODO: Implement

```
`pass
`
```

</details>

<details>

<summary>练习 2: 转换 yaml 字段</summary>

任务：将以下命令转换为 yaml 配置文件。

命令：

```
`bash
llamafactory-cli train \
--model_name_or_path Qwen/Qwen2.5-7B-Instruct \
--dataset alpaca_gpt4_zh \
--template qwen --finetuning_type lora \
--lora_target all --lora_rank 16 --lora_alpha 32 \
--output_dir saves/qwen7b-lora \
--per_device_train_batch_size 2 \
--learning_rate 1e-4 --num_train_epochs 3.0
`
```

验收标准：

- [] 所有字段都正确转换
- [] 格式符合 LLaMA-Factory 规范
- [] 可以直接使用

</details>

课后作业

完成本章后，去 Assignment 12 完成练习：

[Assignment 12](../../assignments/assignment_12/)

下一步

数据从哪来、怎么清洗？Part 13 用手写 MinHash + Data-Juicer 回答（RL 基建见 Part 11）。

[\[← 上一章\]\(01_handwritten_sft_lora.md\)](#) | [\[Part 12 README\]\(README.md\)](#)